

# ARCHITECTURE

## Containers Module - Architecture

**Version:** 2.0.0

**Last Updated:** February 2026

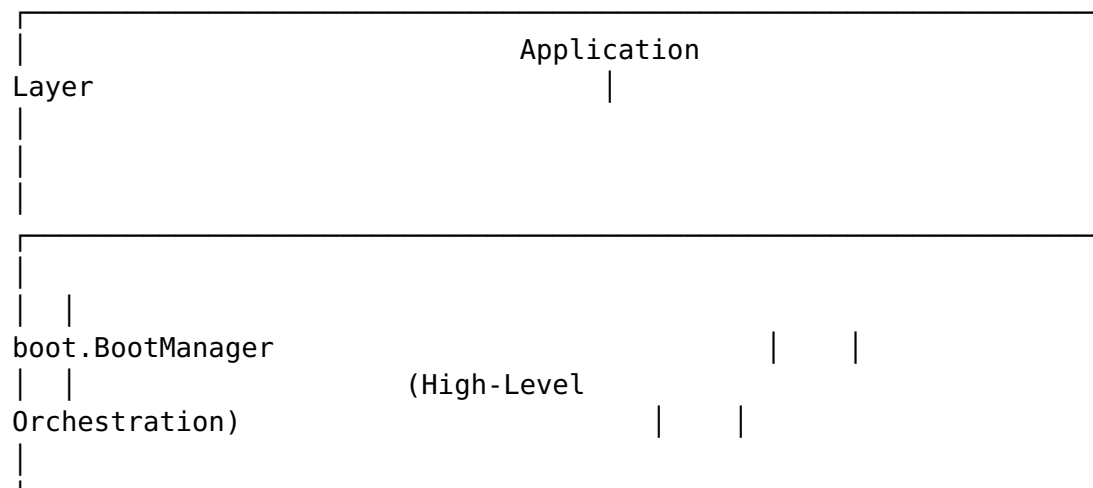
---

### Design Philosophy

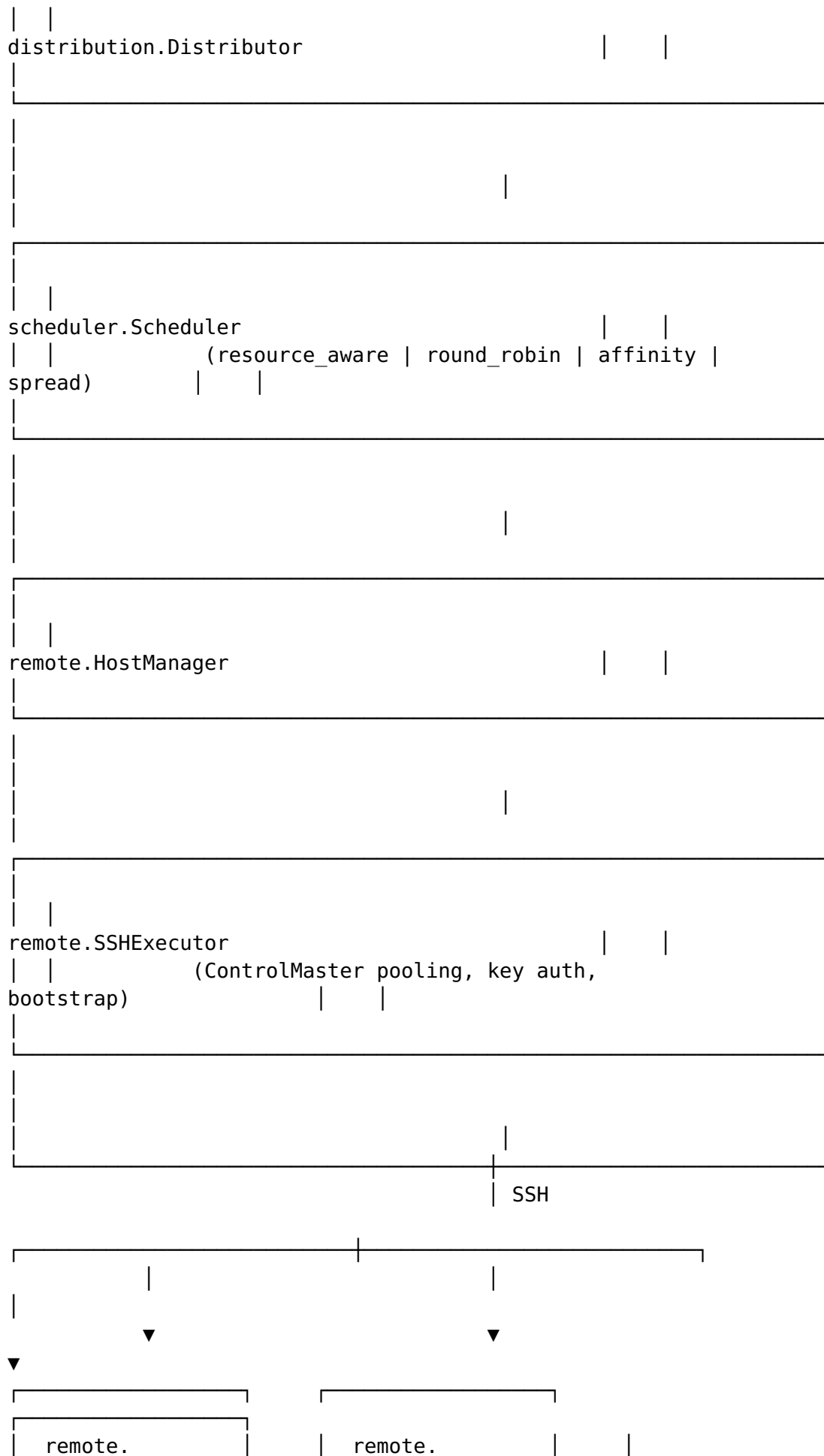
The Containers module provides a **generic, runtime-agnostic** abstraction for container orchestration. It is designed to:

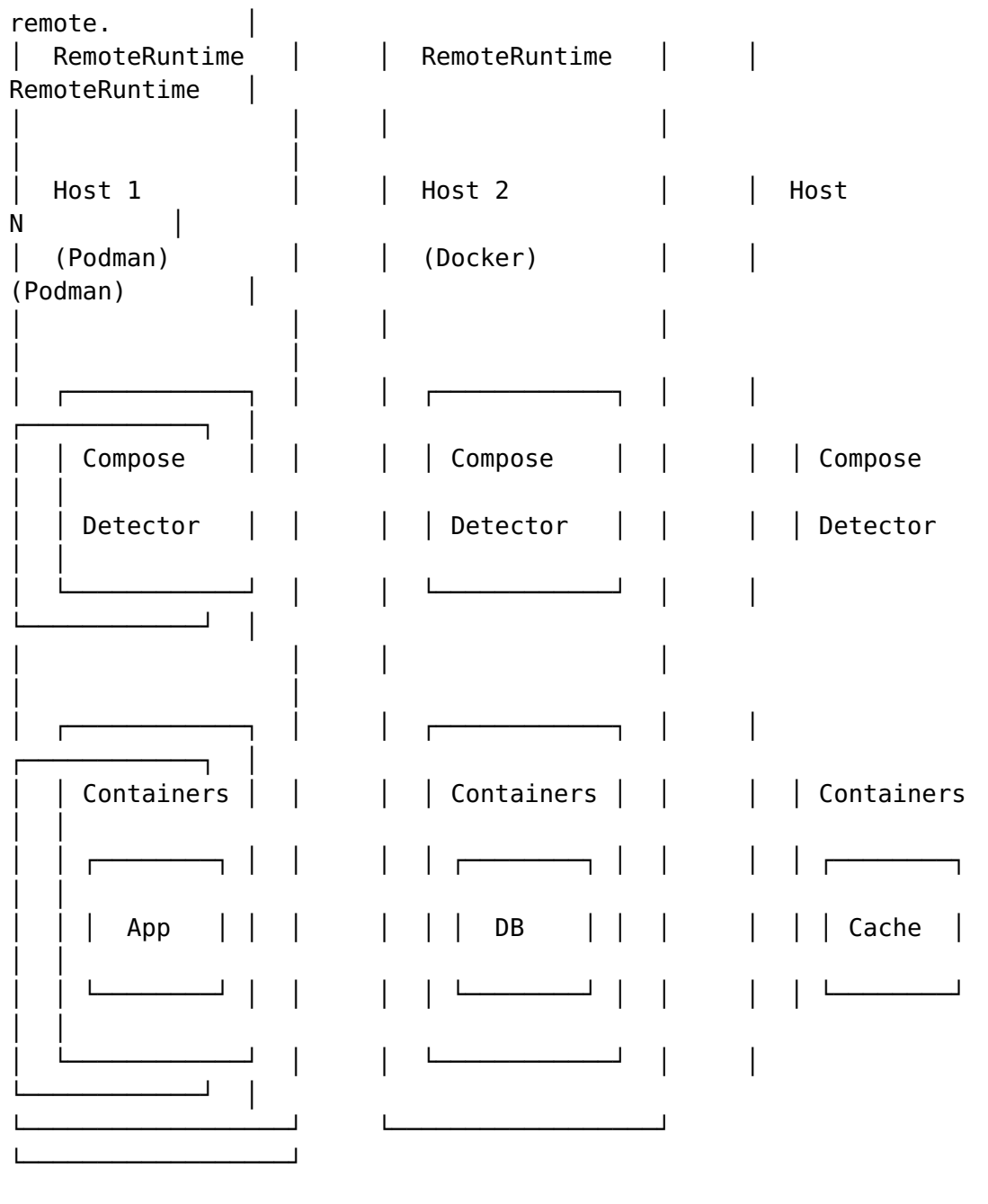
1. **Be reusable** - No dependencies on specific applications
  2. **Support multiple runtimes** - Docker, Podman, Kubernetes
  3. **Enable remote distribution** - Deploy to multiple hosts via SSH
  4. **Be observable** - Health checks, metrics, events
  5. **Be safe** - Thread-safe, proper cleanup, graceful shutdown
- 

### High-Level Architecture

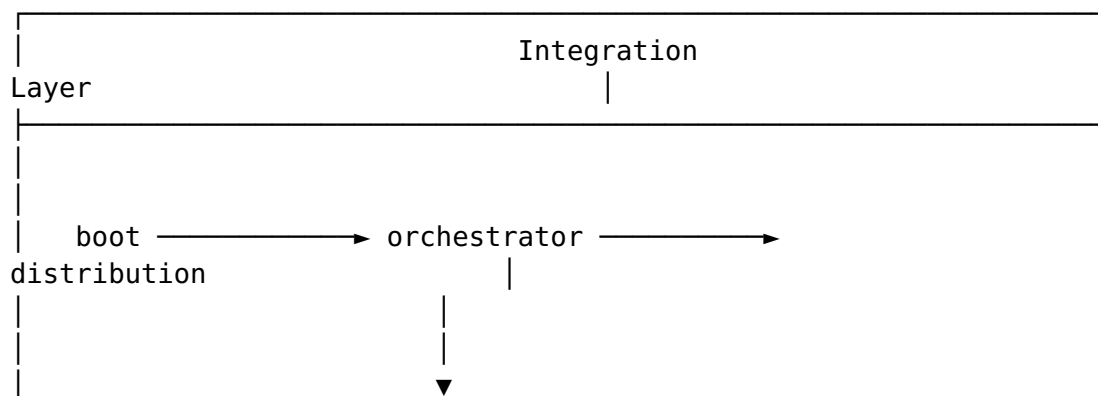


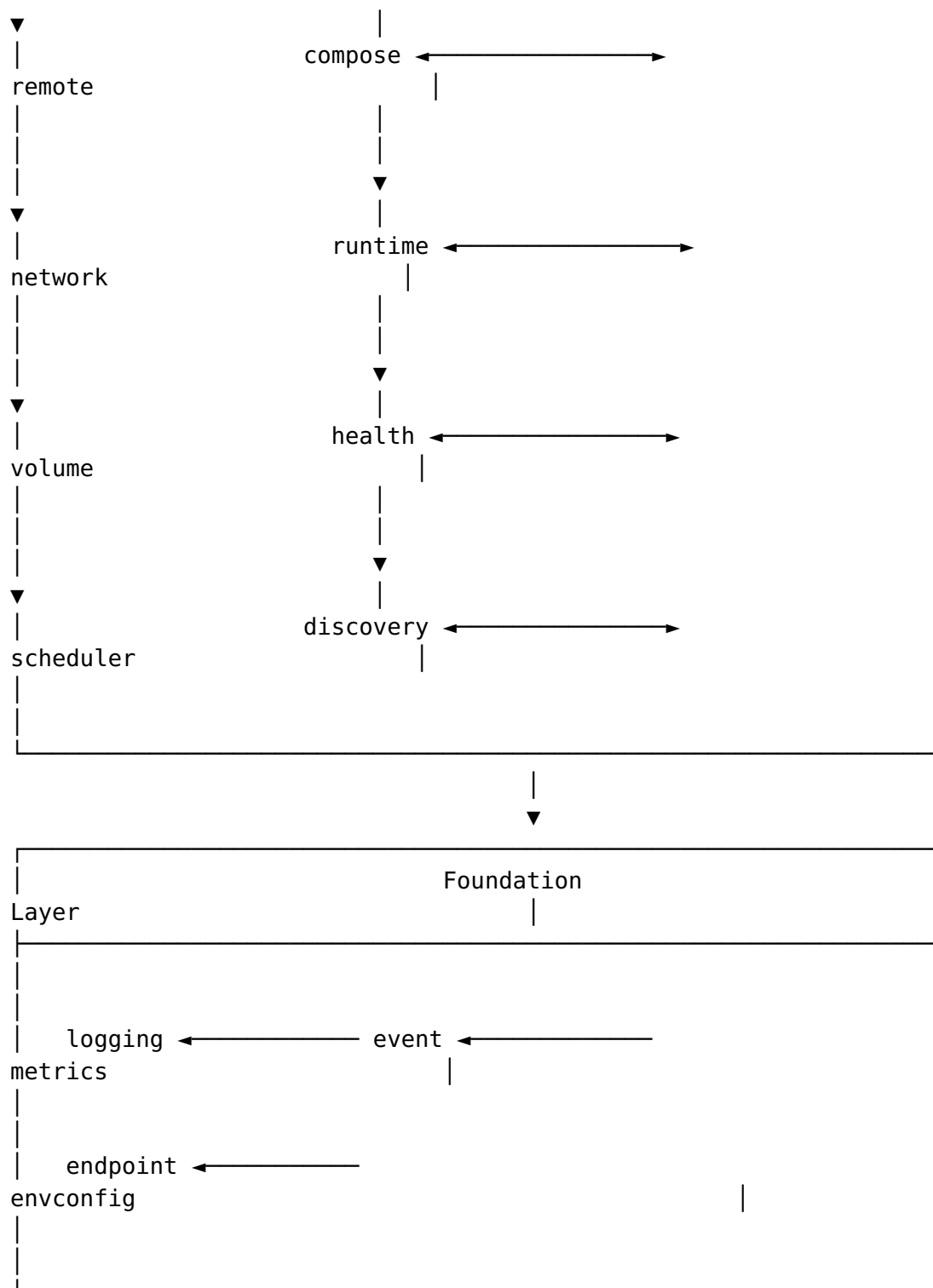






## Package Dependency Graph

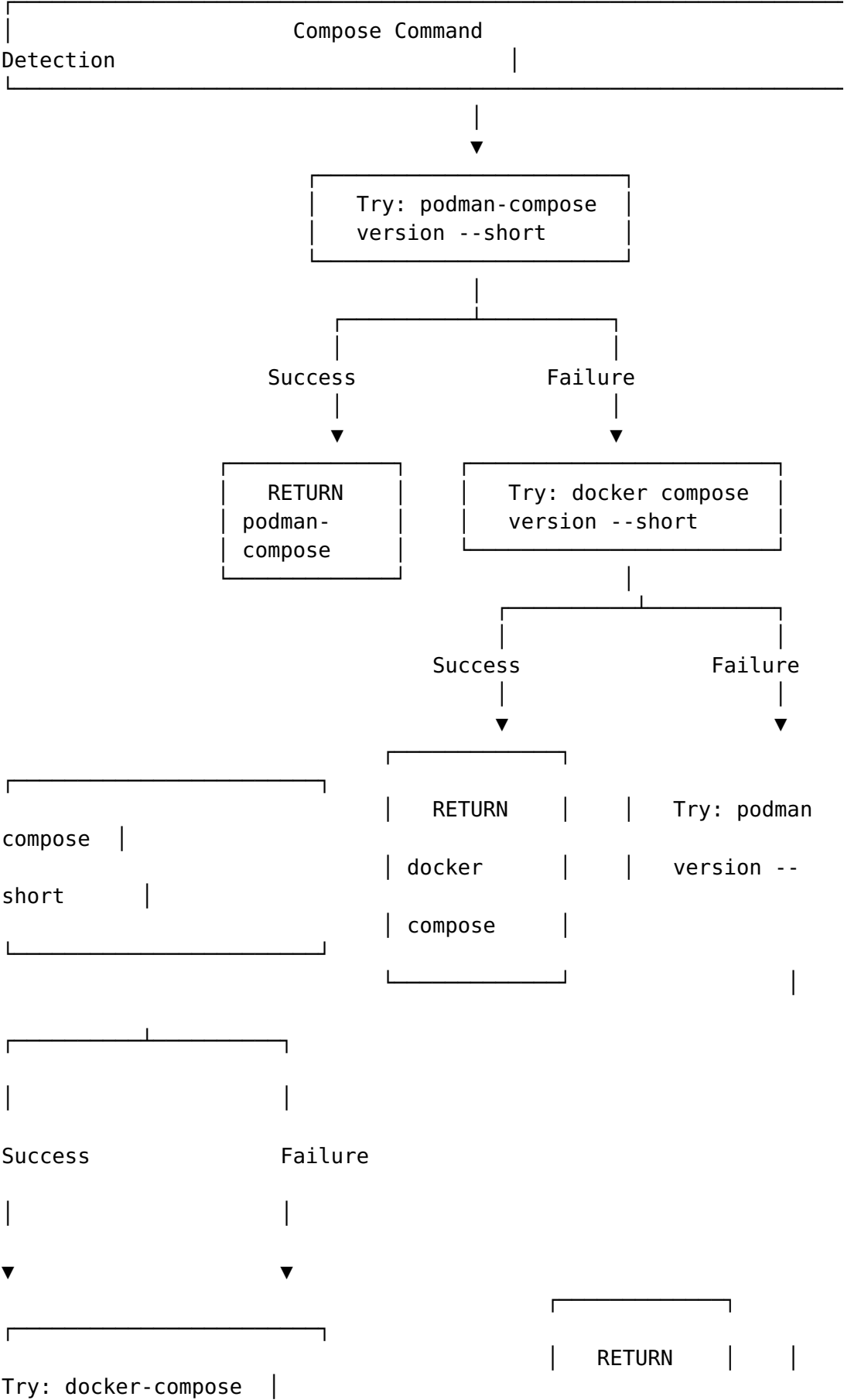


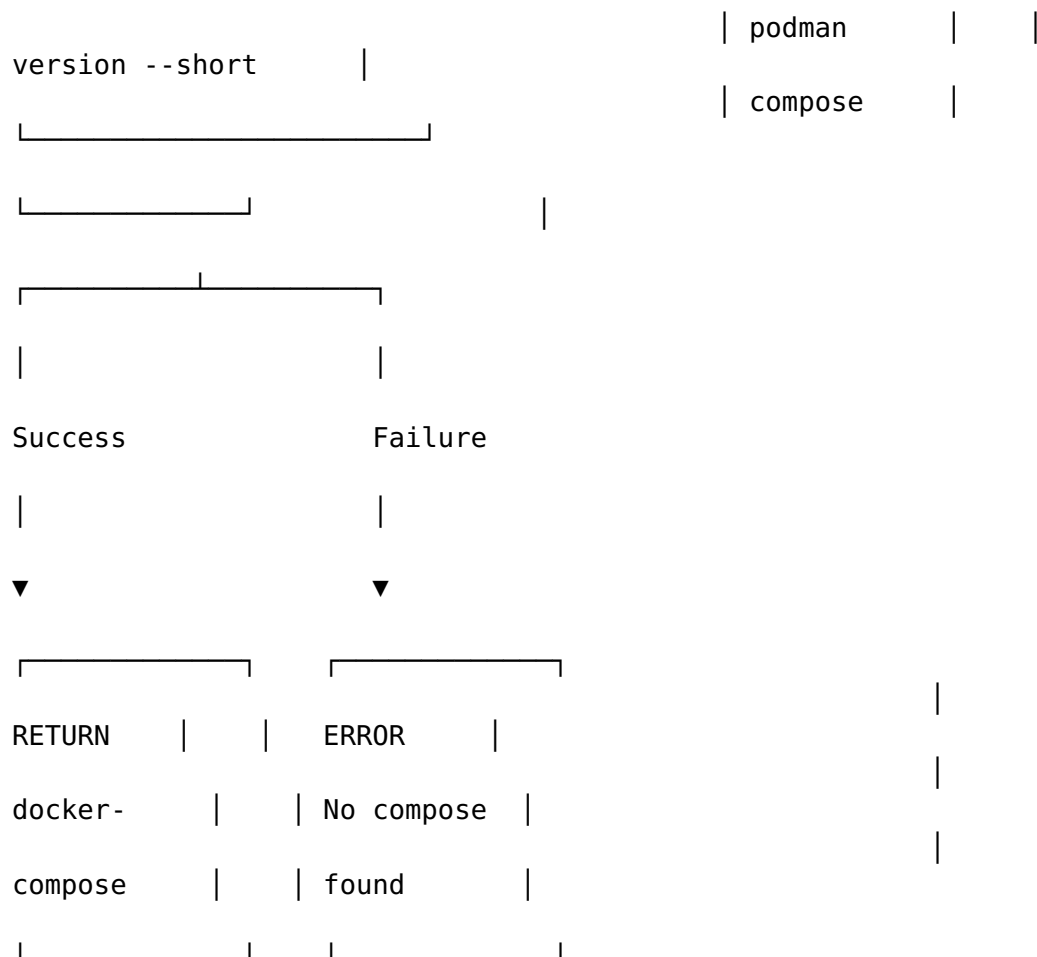


## Dependency Rules

1. **No circular dependencies** - The graph is strictly acyclic
2. **Foundation packages have no internal dependencies**
3. **Integration packages may depend on multiple foundation packages**
4. **boot is the top-level integrator**

# Compose Detection Flow

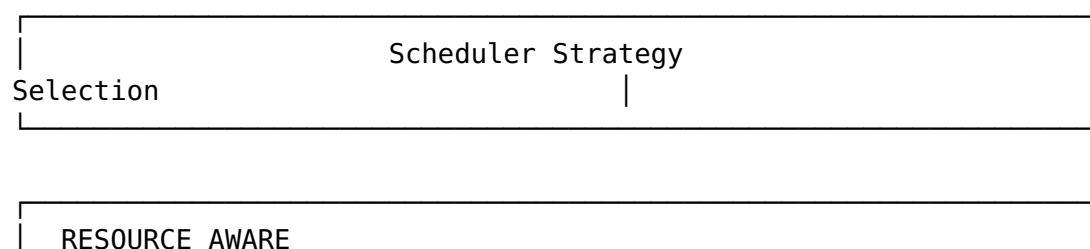




## Why podman-compose is Preferred

| Command           | Issue                                        | Solution                                    |
|-------------------|----------------------------------------------|---------------------------------------------|
| podman-compose    | Delegates to docker-compose v1               | Use podman-compose instead                  |
| docker-compose v1 | Incompatible with Podman (http+docker error) | Install Docker Compose v2 or podman-compose |
| podman-compose    | ✓ Native Podman support                      | Preferred for Podman hosts                  |

## Scheduler Strategy Flow



(Default)

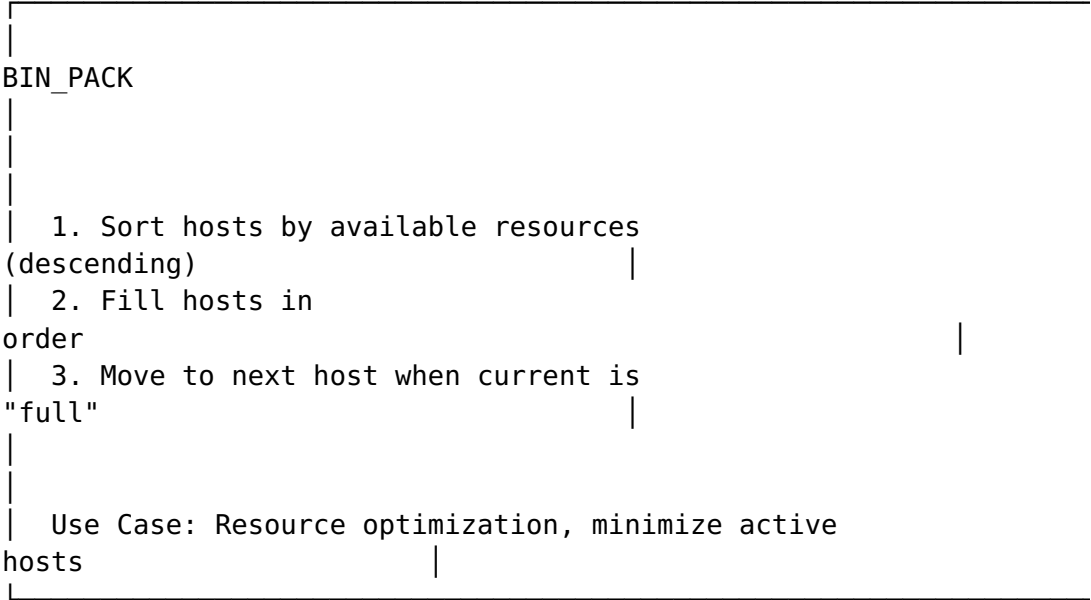
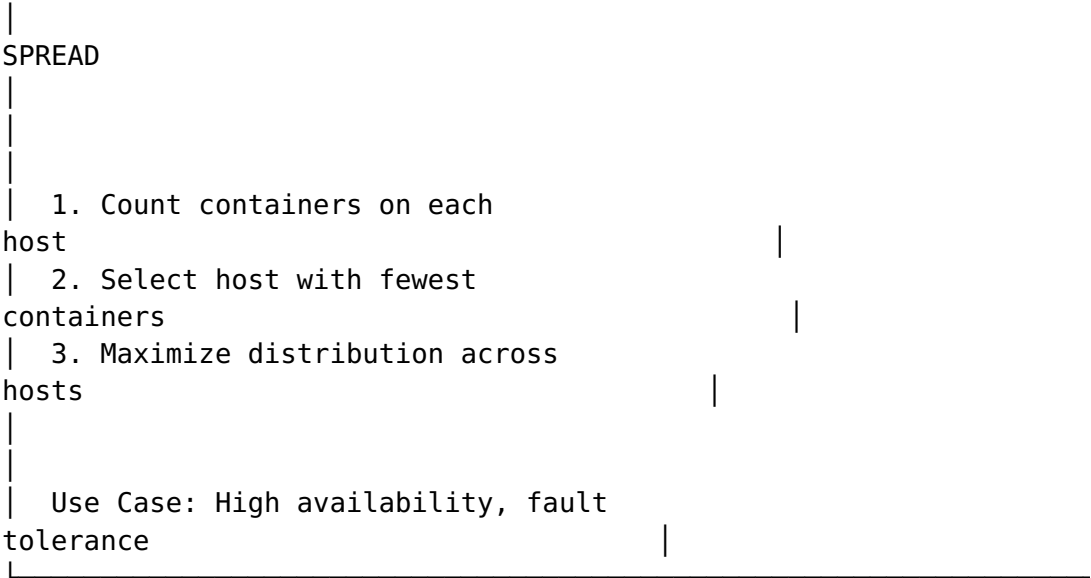
1. Probe each host's CPU, Memory, Disk, Network
  2. Calculate score: CPU(40%) + Memory(40%) + Disk(10%) + Network(10%)
  3. Select host with highest available resources
- Use Case: Mixed workloads, optimal resource utilization

## ROUND\_ROBIN

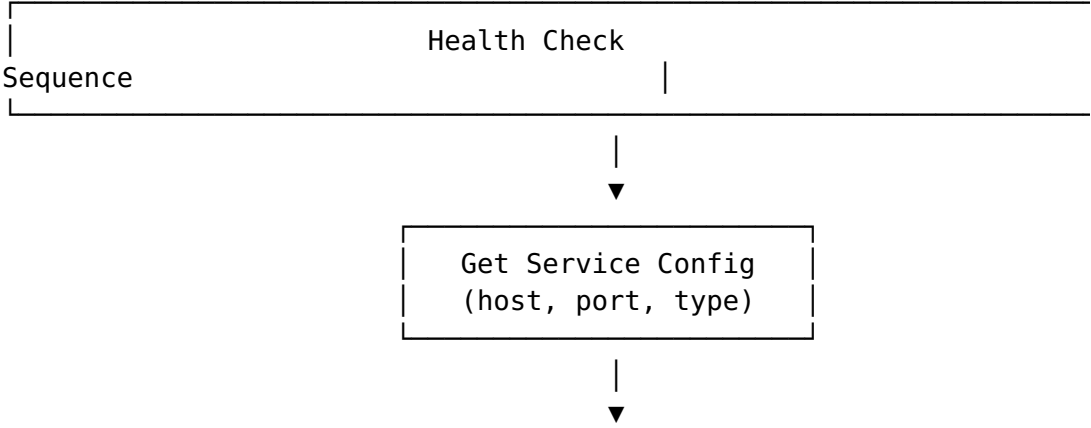
1. Maintain index of last used host
  2. Select next host in rotation
  3. Wrap around when reaching end
- Use Case: Simple load balancing, equal distribution

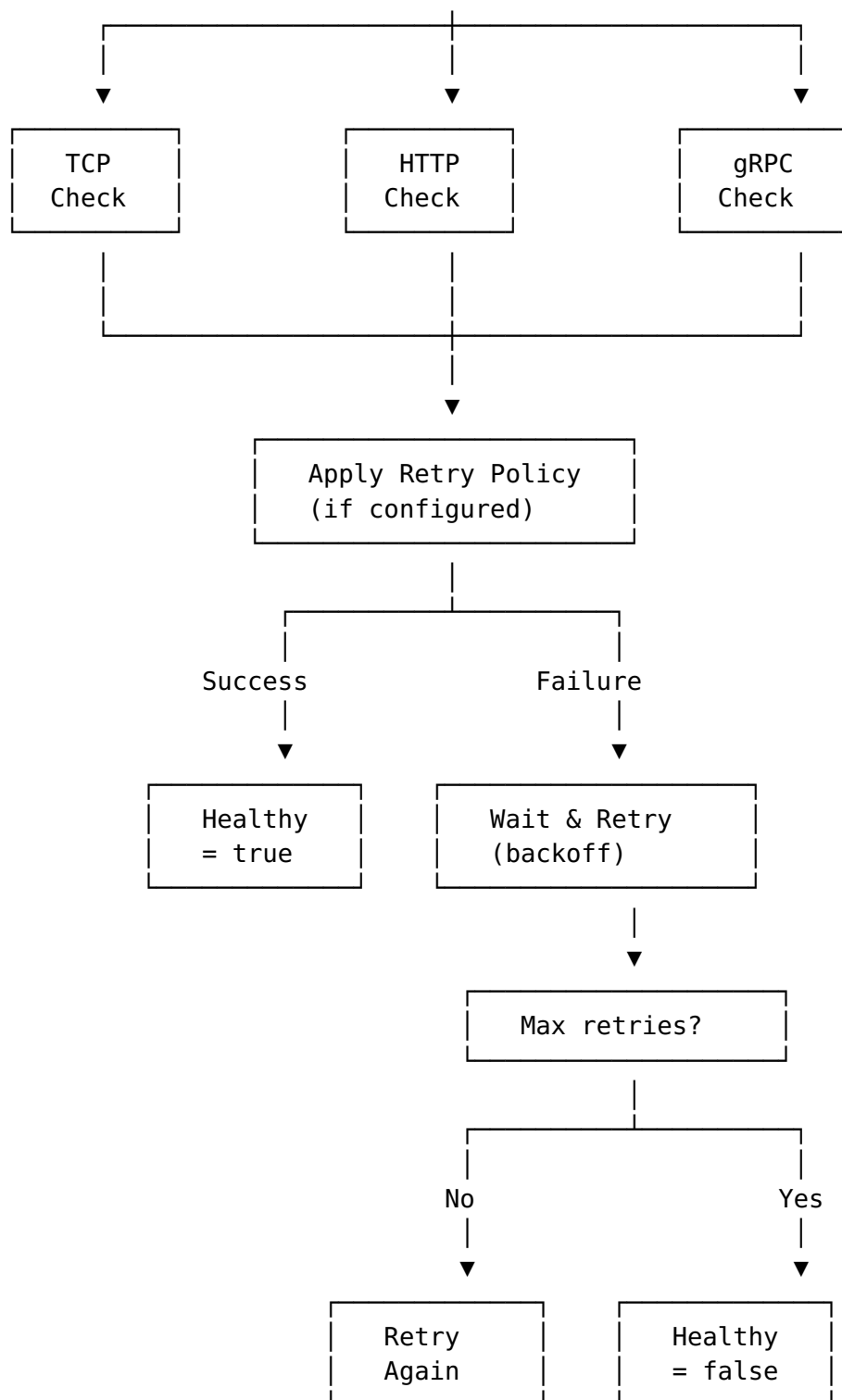
## AFFINITY

1. Match container labels with host labels
  2. Prefer hosts with matching labels
  3. Fall back to other hosts if no match
- Use Case: GPU workloads, storage requirements, architecture matching



# Health Check Flow





---

## Thread Safety Model

| Thread Safety |  |
|---------------|--|
| Guarantees    |  |

| Package            | Mechanism               | Scope              |
|--------------------|-------------------------|--------------------|
| boot.BootManager   | sync.RWMutex            | Service registry,  |
| state              |                         |                    |
| runtime.*Runtime   | sync.Mutex per client   | API calls          |
| compose.*Orchestr. | Stateless               | None needed        |
| health.*Checker    | Stateless               | None needed        |
| remote.HostManager | sync.RWMutex            | Host registry      |
| remote.SSHExecutor | sync.Pool + ControlMstr | Connection pooling |
| scheduler.*        | sync.RWMutex            | Host scores        |
| event.*EventBus    | Channel-based           | Event delivery     |
| monitor.*Monitor   | sync.RWMutex            | Metrics snapshot   |
| lifecycle.*Manager | sync.Mutex + semaphore  | Lifecycle state    |
| distribution.*     | sync.RWMutex            | Workflow state     |

## Design Patterns Used

### Strategy Pattern

- ContainerRuntime interface with Docker/Podman/K8s implementations
- HealthChecker with TCP/HTTP/gRPC/Custom check strategies
- Scheduler with 5 scheduling strategies

### Observer Pattern

- EventBus publishes lifecycle events (started, stopped, health changed)
- Subscribers filter by event type or source

### Factory Pattern

- runtime.AutoDetect() creates the appropriate runtime
- health.NewDefaultChecker() creates a checker with all strategies

### Builder Pattern

- endpoint.NewEndpoint().WithHost().WithPort().Build()
- Fluent API for constructing ServiceEndpoint configurations

### Decorator Pattern

- health.RetryPolicy wraps any health check with retry logic
- Logging wrappers can be applied to any interface

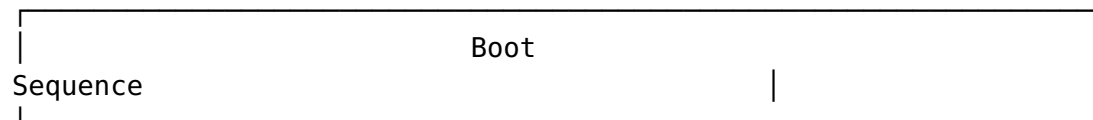
## Functional Options

- `boot.NewBootManager(WithRuntime(), WithLogger(), ...)`
- Extensible configuration without breaking changes

## Proxy Pattern

- `remote.RemoteRuntime` implements `ContainerRuntime` over SSH
  - `remote.RemoteComposeOrchestrator` implements `compose` over SSH
- 

## Boot Sequence



1. DISCOVERY PHASE
  - └─ Probe endpoints with discovery enabled
  - └─ Mark discovered services (skip compose start)
2. GROUPING PHASE
  - └─ Group services by compose file + profile
  - └─ Separate local vs remote services
3. START PHASE (Local)
  - └─ For each compose group:
    - └─ Execute: `docker compose -f <file> --profile <p> up -d`
    - └─ Wait for command completion
  - └─ Collect started services
4. DEPLOY PHASE (Remote)
  - └─ For each remote compose group:
    - └─ Copy compose file to remote host
    - └─ Detect compose command on remote
    - └─ Execute: `podman-compose -f <file> up -d`
    - └─ Collect deployed services
  - └─ Handle failures with fallback
5. HEALTH CHECK PHASE
  - └─ For each enabled service:
    - └─ Execute health check (TCP/HTTP/gRPC)
    - └─ Apply retry policy if configured
    - └─ Record result
  - └─ Aggregate results
6. SUMMARY PHASE
  - └─ Count: started, failed, skipped, remote

- └─ Fail if required services are unhealthy
- └─ Return BootSummary

## Error Handling Strategy

| Error Handling      |                     |                                  |
|---------------------|---------------------|----------------------------------|
| Strategy            |                     |                                  |
| Error Type          | Action              | Recovery                         |
| Runtime unavailable | Warn, continue      | Fall back to compose commands    |
| Compose fail        | Log error, continue | Mark service as failed           |
| Health check fail   | Retry with backoff  | Mark unhealthy after max retries |
| SSH connection fail | Retry with backoff  | Mark host offline, try next host |
| Scheduler fail      | Log, use first host | Fall back to round-robin         |
| Volume mount fail   | Log warning         | Continue without volume          |
| Tunnel fail         | Log, use direct     | Fall back to direct connection   |

## Performance Considerations

### SSH Connection Pooling

| ControlMaster Connection |  |
|--------------------------|--|
| Pool                     |  |

Without ControlMaster:

Command 1: TCP connect + SSH handshake + exec (~500ms)  
Command 2: TCP connect + SSH handshake + exec (~500ms)  
Command 3: TCP connect + SSH handshake + exec (~500ms)  
Total: ~1500ms

With ControlMaster:

Command 1: TCP connect + SSH handshake + exec (~500ms)  
Command 2: Reuse connection + exec (~50ms)  
Command 3: Reuse connection + exec (~50ms)  
Total: ~600ms (60% faster)

# Resource Monitoring

|          |                     |
|----------|---------------------|
|          | Resource Monitoring |
| Overhead |                     |

| Operation         | Frequency | Overhead           |
|-------------------|-----------|--------------------|
| Container list    | 10s       | ~5ms (local)       |
| Container inspect | 10s       | ~2ms per container |
| System metrics    | 10s       | ~1ms               |
| Remote probe      | 60s       | ~50ms (SSH)        |

*Last Updated: February 2026*